# Übungsblatt 3: Interrupts und Timer

**Mikrocomputertechnik 3 · 32 Orientierungspunkte · ca. 60–75 Minuten**

**Musterlösung**

## Aufgabe 1: Drehgeber abfragen (4 Punkte)

Eine Hauptschleife beobachtet einen Drehgeberstatus alle 400 $\mu$s. Die Anforderung lautet: Ereignisse innerhalb von 120 $\mu$s erkennen. Weitere Verzögerungen werden zunächst vernachlässigt.

1. Skizzieren Sie den ungünstigsten Ereigniszeitpunkt zwischen zwei Abfragen. Ist die Frist garantiert? Begründen Sie mit einer Zeitgrenze. (2 P)
2. Der Eingang liefert nur einen ungespeicherten 30 $\mu$s-Impuls. Garantieren Abfragen alle 100 $\mu$s dessen Erkennung? (1 P)
3. Hardware speichert stattdessen ein Bereitschaftsbit bis zur Bearbeitung. Drei Impulse treffen vorher ein. Welche Information bleibt erhalten, welche nicht? (1 P)

**Musterlösung 1**

1. Abfragen bei null und 400 $\mu$s, Ereignis unmittelbar nach null. Die Wartezeit kann nahezu 400 $\mu$s betragen: 120 $\mu$s sind nicht garantiert. (2 P)
2. Nein. Ein 30 $\mu$s-Impuls kann vollständig zwischen zwei Beobachtungen liegen. (1 P)
3. Erhalten bleibt „mindestens ein Ereignis“. Die Anzahl drei und zugehörige einzelne Daten sind aus diesem Bit allein nicht rekonstruierbar. (1 P)

## Aufgabe 2: Eintritt und Fortsetzung (6 Punkte)

RV32-Lehrmodell: ein Hart im Machine Mode, keine Weiterleitung an andere Privilegstufen und keine verschachtelten Interrupts. Ein Hart ist ein Hardware-Ausführungskontext. Bei einem Trap erhält `mcause` den Wert `0x8000000B`. Bit 31 unterscheidet Interrupt und Exception, Code elf bezeichnet Machine External.

1. Was sagt der Rohwert aus? Ist damit bereits eine externe Geräte-ID bekannt? (2 P)
2. In einem separaten Zustand wartet eine Timeranforderung. Timerfreigabe MTIE ist eins, globale Freigabe MIE null. Wird der Timer-Trap angenommen? Läuft die Hardwarezeit weiter? (2 P)
3. Welche Seite erhält jeweils den Zustand: Hardware oder Software? Gesucht sind die Fortsetzungsadresse in `mepc` und ein vom Handler verändertes, im Hauptprogramm noch benötigtes allgemeines Register. (1 P)
4. Vectored-Modus: BASE `0x80400400`, Interrupt-Code elf. Berechnen Sie den ersten Einsprung mit BASE $+ 4c$. (1 P)

**Musterlösung 2**

1. Interrupt-Bit eins, Code elf: externe Maschinen-Interruptkategorie. Die konkrete Geräte-ID fehlt und wird erst per Controller-Claim ermittelt. (2 P)
2. Kein betrachteter Timer-Trap bei MIE null. Die Zeitquelle läuft weiter. (2 P)
3. Hardware hält die Fortsetzungsadresse in `mepc` fest. Software muss das benötigte allgemeine Register sichern und wiederherstellen. (1 P)
4. $4 \cdot 11 = 44 =$ `0x2C` Byte, Einsprung `0x8040042C`. Auch dort muss noch die externe Gerätequelle ausgewählt werden. (1 P)

## Aufgabe 3: Gerätebehandlung vollständig machen (6 Punkte)

Ein PLIC-Kontext, alle Quellen wartend und freigegeben, Schwellenwert null. PLIC bedeutet Platform-Level Interrupt Controller. Höhere Priorität gewinnt, bei Gleichstand die kleinere Geräte-ID.

| ID | Quelle | Priorität |
|---|---|---|
| 9 | serielle Schnittstelle | 4 |
| 12 | Beschleunigungssensor | 8 |
| 5 | Drehgeber | 8 |

1. Nennen Sie die ersten beiden Claim-IDs. Nach jedem Claim wird vollständig behandelt und abgeschlossen; keine neuen Ereignisse. Begründen Sie. (2 P)
2. Was ändert Claim am Controller? Ist damit die Ursache im Gerät beseitigt? (1 P)
3. Ein fehlerhafter Handler führt nur Claim und Complete aus. Das Gerät hält seine Level-Anforderung weiter aktiv. Ergänzen Sie den fehlenden Schritt und erklären Sie die mögliche Folge. (2 P)
4. Ein späterer Claim liefert null. Welche Gerätebearbeitung und welches Complete sind dann vorgesehen? (1 P)

**Musterlösung 3**

1. ID fünf, danach zwölf. Beide haben Priorität acht, im Gleichstand gewinnt fünf. (2 P)
2. Claim übernimmt eine Quelle und löscht atomar deren PLIC-Pending. Die Geräteursache kann weiter bestehen. (1 P)
3. Zwischen Claim und Complete Daten beziehungsweise Status nach Gerätevertrag bearbeiten und die Ursache passend quittieren. Eine weiter aktive Level-Ursache kann nach Complete erneut gemeldet werden. (2 P)
4. Keine Gerätebearbeitung und kein Complete für ID null. (1 P)

## Aufgabe 4: Ein 2,5-ms-Sollraster (7 Punkte)

Hardwarezeit `mtime`: zwölf MHz, Prozessortakt: 96 MHz. Sollperiode 2,5 ms, nächster Solltermin $d = 240000$ Timer-Ticks. `mtimecmp` enthält den Vergleichstermin. Zeitwerte werden konsistent gelesen und geschrieben; kein Zählerumlauf oder arithmetischer Überlauf.

1. Bestimmen Sie die Sollperiode in Timer-Ticks. Würde eine Änderung nur des Prozessortakts auf 48 MHz diesen Wert ändern? (2 P)
2. Die Behandlung beginnt bei now 249000. Vergleichen Sie now plus Periode mit dem nächsten absoluten Rastertermin, jeweils in Ticks und Millisekunden. (2 P)
3. Separat now 346000: Wie viele Solltermine sind fällig? Welcher nächste Termin liegt auf dem Raster und strikt in der Zukunft? (2 P)
4. Der Sensor hält nur seinen neuesten Messwert bereit. Lassen sich die verpassten historischen Messwerte durch unmittelbar wiederholtes Lesen nachholen? Begründen Sie. (1 P)

**Musterlösung 4**

1. $12 \cdot 10^6 \cdot 0{,}0025 = 30000$ Ticks. Die separat vorgegebene Timerfrequenz bleibt unverändert. (2 P)
2. Relativ: $249000 + 30000 = 279000$ Ticks $= 23{,}25$ ms. Absolut: $240000 + 30000 = 270000$ Ticks $= 22{,}5$ ms. Die relative Planung übernimmt die Eintrittsverspätung. (2 P)
3. $k = \lfloor (346000 - 240000)/30000 \rfloor + 1 = 4$. Fällig sind 20, 22,5, 25 und 27,5 ms. Nächster Termin: $240000 + 4 \cdot 30000 = 360000$ Ticks $= 30$ ms. (2 P)
4. Nein. Wiederholtes aktuelles Lesen rekonstruiert keine zuvor ungespeicherten Werte. Erforderlich wäre ihre rechtzeitige Erfassung und Speicherung. (1 P)

## Aufgabe 5: Zwei verschiedene Reaktionsfristen (4 Punkte)

1. Prozessor 80 MHz, 500 vollständige ISR pro Sekunde, je 640 Prozessorzyklen einschließlich Ein- und Austritt. Berechnen Sie Dauer und mittleren Prozessorzeitanteil. (2 P)
2. Separates Reaktionsmodell: Bis zum Beginn einer eigenen Behandlung fallen höchstens $60\,\mu$s Blockierung und $4\,\mu$s Eintritt an. Das Sichern der Daten benötigt weitere $6\,\mu$s. Die spätere Auswertung wartet höchstens $120\,\mu$s und dauert $40\,\mu$s. Zeichnen Sie die Zeitkette. Wann sind Daten gesichert, wann ist das Ergebnis fertig? (1 P)
3. Für welchen dieser Endpunkte ist eine Frist von $100\,\mu$s eingehalten? Erklären Sie, warum der mittlere Zeitanteil aus Teil 1 allein nicht genügt. (1 P)

ISR bedeutet Interrupt Service Routine. Alle Zeiten sind Lehrannahmen; zusätzliche Verzögerungen sind im jeweiligen Modell ausgeschlossen.

**Musterlösung 5**

1. $640/(80 \cdot 10^6) = 8\,\mu$s. $500 \cdot 8\,\mu\text{s/s} = 0{,}004 = 0{,}4\,\%$. (2 P)
2. Daten gesichert nach $60 + 4 + 6 = 70\,\mu$s. Ergebnis fertig nach $70 + 120 + 40 = 230\,\mu$s. (1 P)
3. Nur die Datensicherung erfüllt $100\,\mu$s. Mittlere Auslastung begrenzt weder maximale Blockierung noch spätere Wartezeit. (1 P)

## Aufgabe 6: Ereignisse sicher übernehmen (5 Punkte)

Ein Hart, keine verschachtelten Interrupts. Ein Zähler hält die Anzahl noch zu bearbeitender Ereignisse fest und steht anfangs auf zwei. Einzelne Wortzugriffe sind im Plattformmodell unteilbar, die folgende Gesamtfolge nicht.

| Schritt | Hauptprogramm | ISR |
|---|---|---|
| 1 | liest Zähler zwei in einen lokalen Merker | |
| 2 | | erhöht den Zähler auf drei |
| 3 | beendet ein Ereignis und schreibt Merker minus eins zurück | |

1. Welcher Zählerstand entsteht, welcher wäre nach einem bearbeiteten Ereignis richtig? Behebt `volatile` allein den Fehler? (1 P)
2. Skizzieren Sie stattdessen eine kurze geschützte Folge, die den gesamten Zählerstand lokal übernimmt und den gemeinsamen Zähler auf null setzt. Wo liegt die längere Verarbeitung? (2 P)
3. Warum wird danach der vorherige Freigabestand wiederhergestellt, statt grundsätzlich alle Interrupts einzuschalten? (1 P)
4. Die Anforderung ändert sich: Alle einzelnen Messwerte müssen erhalten bleiben. Reicht ein Ereigniszähler? Begründen Sie die benötigte Datenablage. (1 P)

**Musterlösung 6**

1. Es entsteht eins statt zwei: Ein Ereignis geht in der Zählung verloren. `volatile` macht die mehrschrittige Änderung nicht atomar. (1 P)
2. Vorherigen Freigabestand sichern, betreffende Interrupts sperren, Zähler lokal übernehmen und auf null setzen, Freigabestand wiederherstellen. Die lokal übernommene Anzahl wird danach außerhalb der Sperre verarbeitet. (2 P)
3. Eine bereits zuvor gewollte Sperre darf nicht unerwartet aufgehoben werden. (1 P)
4. Nein. Ein Zähler erhält nur die Anzahl, nicht die Werte. Benötigt wird ein ausreichend großer Datenpuffer mit konsistenter Übergabe und einer Regel für Überlast beziehungsweise Überlauf. (1 P)

## Optional: Terminrechnung als C-Funktion prüfen

Implementieren Sie die Zukunftsterminrechnung aus Aufgabe 4 als reine C-Funktion. Prüfen Sie bei Starttermin 240000 und Periode 30000 die now-Werte 239999, 240000 und 346000. Erwartet: 240000, 270000 und 360000. Ein Simulator ist dafür nicht nötig. Die Hardwarezugriffe aus den Vertiefungsfolien sind keine Voraussetzung für diese Prüfung.